Akai | SE3 Projekt

DOKUMENTATION

Beschreibung:

Bei unserem SE3-Projekt “Akai” handelt es sich um ein 2D-Platformer-Metroidvania Game im Pixel-Art Stil. Der User versetzt sich in die Rolle des furchtlosen roten Pandas “Akai” und hilft ihm dabei, während seinem Abenteuer Gegner zu besiegen und neue Fähigkeiten freizuschalten. Für die Umsetzung des SE3 Projektes wurden folgende Technologien verwendet: C# und die Game Engine Unity.

Wie die Applikation gestartet wird:

Es wird ein Build vom Projekt gemacht und eine .exe Datei erstellt. Diese kann ausgeführt werden.

Anforderungen:

Als Anforderungsanalyse haben wir uns folgende Aspekte für unser Software-Projekt anhand einer User Story überlegt:

- Ich als Spieler möchte springen und bewegen, um Hindernisse zu überwinden.
- Ich als Spieler möchte mehrere Leben haben, um nicht sofort zu sterben.
- Ich als Spieler möchte Gegner schlagen, um diese zu besiegen.
- Ich als Spieler möchte eine Währungssache sammeln, um Ausrüstung freizuschalten.
- Ich als Spieler möchte die Möglichkeit haben das Spiel zu pausieren.
- Ich als Spieler möchte gemeinsam mit anderen Freunden spielen können

Diese Anforderungen wurden wiederum technisch wie folgt umgesetzt:

Fachliche Anforderungen	**Technische Umsetzung**
Beweglichkeit	
mehrere Leben	
Gegner schlagen	PlayerController, EnemyController, BigEnemyController
Geld/Währung einsammeln	Interactables und UIController (CoinController)
Spiel pausieren	Implementierung vom Skript Pausenmenü
Multiplayer-Mode	Umsetzung mit NetCode

Projektverwaltung

Für die Priorisierung unserer Anforderungen und Aufgaben des Projektes haben wir uns für ein Ticketing-System entschieden. Hierbei haben wir die Plattform Jira verwendet. In einem Kanban-Board wurden verschiedene Kategorien (Code-Aufgaben, In Progress, Done etc.) erstellt, in welche wir unsere Tickets und Boards entsprechend platziert haben.

In den einzelnen Tickets wurden nähere Informationen zur jeweiligen Code-Aufgabe notiert. So auch die fachlichen und technischen Anforderungen.

Ablauf

1. Eine Anforderung lautet z.B. "*Währung sammeln"*
2. Der entsprechende KAN-Task besitzt eine Nummer (KAN 62):
3. In der Task folgt eine Beschreibung, welche der fachlichen Anforderung entspricht. In diesem Fall: *Ich als Spieler möchte eine Währungssache sammeln, um Ausrüstung freizuschalten.*
4. Diese werden mit den technischen Anforderungen verbunden, welche unter den Akzeptanzkriterien aufgelistet sind. Beim Sammeln von Münzen eignet es sich die Anforderung anhand von einem Counter umzusetzen, welche sich erhöht, sobald der Player in Berührung mit einer Münze kommt.

Timeline

In der Timeline wurden die wichtigsten Milestones unseres Projektes festgelegt. Für weitere Details siehe Link:

https://akaisadventure.atlassian.net/jira/software/projects/KAN/boards/1/timeline

Beim Erstellen von git commits wurden bei den wichtigsten Änderungen und bei der Fertigstellung dieser Aufgaben Kommentare hinzugefügt. In diese sind die dazugehörige KAN-BAN Nummer notiert.

Softwarearchitektur

Aufgrund der resultierenden Anforderungen haben wir uns dazu entschieden, uns nach den Prinzipien des Schichtenmodells zu orientieren. Dabei haben wir die Kriterien an unser Software-Projekt angepasst. Das Projekt setzt sich aus den Schichten "Controller", "Persistence" und "Data" zusammen.

Controller:

Die Controller sind maßgeblich für die Manipulation und Verhalten der Gameobjekte, der Szenen, das Aufsetzen des Multiplayer-Modes, sowie alle weiteren GUI-Elemente des Projektes verantwortlich. Sie sind sowohl für die Steuerung der Player und Enemies wie auch für die Kamera und alle UI-Elemente da. Sie kriegen die Daten aus dem Data-Layer.

In der Persistenzschicht werden wichtige Daten wie Spielerstand erfasst und gespeichert.

Zu guter Letzt werden im Data Layer alle relevanten Daten zu den wichtigsten Gameobjekten “Player” und “Enemy” sowie wichtige Informationen zu den PlayerSettings und des Spiels gespeichert.

Packages: durch namespaces umgesetzt

SOLID Principles:

S - Single Responsibility: Statt große Klassen mit mehreren Funktionen zu bauen, die eventuell auch zu sehr voneinander abhängig werden können, so dass ein Big ball of mud entsteht sind die Klassen nur für bestimmte Funktionen verantwortlich. In unserem Projekt wurde dies durch den Einsatz von Interfaces und Factories realisiert, aus denen wiederum weitere Klassen abgeleitet wurden. Die Verantwortung einer jeden Klasse ist auch durch ihre Namensgebung strukturiert. So ist der CharacterController prinzipiell für die Logik der Bewegung von den unterschiedlichen Gameobjekten Player und Enemy zuständig. Die eigentliche Logik wurden in den entsprechenden Klassen Player und Enemies umgesetzt. Ebenfalls haben wir einen EventHolder, in dem Actions definiert sind (ähnlich wie Events), die von Controllern ausgelöst werden können, damit die Modularität jedes Controllers gewährleistet ist.

Die Single Responsiblity Prinzipien wurden nicht nur auf die Game-Logik angewendet, sondern auch auf andere Bestandteile wie die UIController erweitert. Statt alles in einem C# Skript zu realisieren, sind die Klassen klar modularisiert. .

Factories: Das Erstellen von Enemies wird in der EnemyFactory Klasse realisiert. Dort kann man zwischen SmallEnemies, Enemies und BigEnemies unterscheiden.

Data Ordner: Der Data Ordner besitzt eine klare Verantwortung, da diese für die Speicherung wichtiger Daten zuständig ist.

O - Open for extension and closed for modification: *Der EventHolder ist eine* Klasse, die anhand von Methoden erweitert wird. Die Event-Methoden können wiederum in den anderen Klassen verwendet werden. So werden in der eigentlichen EventHolder Klasse die Methoden nicht modifiziert.

L - Liskov Substitution: Liskov Substitution erweitert das Open Closed Prinzip. Im Falle unseres Game-Projektes ist dies vor allem durch das Interface des CharacterControllers realisiert worden. Der EnemyController umfasst alle Daten eines regulären Gegners während BigEnemyController eine Erweiterung bildet und neue Features einführt wie einen größeren Gegner zu kreieren, der schwieriger zu besiegen ist und zusätzliche Fähigkeiten besitzt wie z.B. kleine Enemies zu spawnen.

I - Interfaces Segregation: In unserem Projekt werden unterschiedliche Interfaces für ihre verschiedene Zwecke verwendet. So existiert z.B. ein Interface, wovon sowohl Player als auch Enemy erben (Die und TakeDamage Methoden). Dann gibt es noch ein Interface, welches für die persistente Erfassung der Spieledaten zuständig ist, wobei die genaue Implementierung in der DataPersistenceManager Klasse umgesetzt wird.

D - Dependency Inversion: High level modules shouldn’t rely on low level modules. Dies werden durch Interfaces, Prefabs sowie dem EventHolder realisiert.

Git:

Zu Beginn des Projektes haben wir auf eine trunkbasierte Entwicklung gesetzt, welches, aufgrund der verwendeten Game Engine Unity, zügig zu eine git-flow basierten Entwicklung wechselte. Beim pushen der gleichen Scene auf git kann es zu Konflikten kommen. Deshalb wurden die Features auf eigens dafür angelegte Branches entwickelt, während die main scene von möglichen Konflikten unberührt bleibt. Die main branch wurde dabei im Laufe des Projektes mehrmals mit dem mergen der neuen Features geupdated.

Für die Umsetzung und Verknüpfung fachlicher und technischer Anforderungen haben wir das Projektmanagement-Tool Jira verwendet. In der einzelnen Task wurden die entsprechenden Anforderungen notiert. Je nach Stand der Task wurde sie in eine passende Kategorie wie “In Progress” verschoben. Nach der Implementierung eines Features wurde dies auf Git gepushed und anschließend mit einem Kommentar versehen, der die Nummer der Kan-Aufgabe enthält.

Änderungen, die Codefehler beheben, wurden in den commit messages immer mit dem Schlüsselwort “fix” notiert.

Die wichtigsten Anforderungen wurden zusätzlich getaggt und die Änderungen in einer Changelog notiert, die regelmäßig geupdatet und versioniert wurde. Näheres dazu unter folgendem Link: https://gitlab.mi.hdm-stuttgart.de/lg100/akai/-/blob/main/CHANGELOG.md?ref_type=heads

Test and Build Management

Für das Testing unseres Game-Projekts wurde das Unity Testing Framework eingesetzt.
Dabei gibt es zwei verschiedene Arten von Testings. Einmal Tests im “EditMode” und einmal “PlayMode”, die jeweils beide eigene Assembly Definitionen brauchen und wiederum mit der Assembly Definition der Scripts verknüpft sind, damit die Testfunktionen auf die Klassen des Projekts zugreifen können.
EditMode entspricht dem UnitTesting, wo einzelne Funktionen und Methoden gekapselt und getestet werden. In unserem Game Projekt ist dies jedoch leider beschränkt möglich, da das Spiel interaktiv ist und bestimmte Bedingungen davor ausgelöst werden müssen, um Methoden zu aktivieren. Hierfür haben wir die PlayMode Tests verwendet.
Die einzelnen Methoden unseres Projektes sind vor unnötigem Zugriff geschützt, weshalb das Testing in bestimmten Fällen aufgrund des “private” modifiers beschränkt ist. Die vorhandenen Tests beziehen sich auf Methoden, die public deklariert sind und auf Tests die sicherstellen, dass das Unity Framework “NetCode” einwandfrei für das Networking funktioniert

CI/CD

Für was brauchen wir CI CD Pipelines?

CI/CD reduziert Code Fehler und Defekte und ist somit entscheidend für jeden DevOps-Arbeitsablauf. Dabei handelt es sich um die Praxis, Code-Änderungen häufig zu integrieren, Code-Integrationen zu automatisieren und die endgültigen Builds automatisch an Endbenutzer freizugeben. Es wird also das Projekt ständig gebuildet, getestet und deployed. Man richtet sich Build-Konfigurationen für jede Plattform, wie z.B. Windows, Linux, Android, MacOS oder IOS, ein.

Wie setzt man die CI für Unity auf?

Die Webseite game.ci bietet schon ein fertiges Projekt an, von dem man sich bedienen kann. Man übernimmt dabei den Ordner ci, der ins Root-Verzeichnis deines Projektes kommt.
Dieser beinhaltet alle Shell Befehle die Unity benötigt, um nachzuprüfen, ob ein Lizenz-Key vorhanden ist, wie die Pipeline seinen Aktivierungs-Key bekommt, sowie die Pipeline in Unity builden oder testen kann. Zusätzlich fügt man den Editor Ordner in das Verzeichnis Assets/Scripts hinzu. Dort befindet sich ein Script namens “BuildCommand”, welches Unity benötigt, um ein Projekt builden zu können. In diesem Script werden sogenannte “Building Paths” gesetzt. Zusätzlich werden Keys für Android oder IOS zugewiesen. Zuletzt wird die “.gitlab-ci.yml” Datei über das Terminal ins Project Root-Verzeichnis kopiert.

1. .gitlab-ci.yml
   In der Datei .gitlab-ci.yml lässt sich folgendes definieren:

   Die Aufgaben, die man abschließen möchte. Zum Beispiel das Testen und Bereitstellen der Anwendung.
   Andere Konfigurationsdateien und Vorlagen, die man einbinden möchte.
   Abhängigkeiten und Caches.
   Die Befehle, die sequentiell ausgeführt werden sollen und diejenigen, die parallel ausgeführt werden.
   Den Speicherort, an dem die Anwendung bereitgestellt werden soll.
   Ob die Scripte automatisch oder manuell ausgeführt werden sollen.

   Anschließend noch die eigene Unity Version, z.B. 2022.3.12f1, in die YAML-Datei eintragen, sowie in die Variable “BUILD_NAME” das Projekt benamen, z.B. Akai.

   Drei essenzielle stages die in der .gitlab-ci.yml sind:
   - prepare
   - build_and_test
   - deploy

Durch Variablen wie “only” kann man Bedingungen setzen, wann diese ausgeführt werden. Z.B. Deploy sollte nur ausgeführt werden, wenn es einen Merge auf die main bzw. default Branch gibt.

Wenn alles in der Datei angelegt ist, wird alles ins Repository gepusht.

2. Nach dem push wurde eine Pipeline in GitLab angelegt die auch dort einsehbar ist.
Der Push ist noch geblockt, da noch ein paar Variablen fehlen, um den ersten Job auszuführen.
Bsp.:
Der erste Job innerhalb der Pipeline ist stets „get-activation-file“. Dieser wird geblockt.
Dafür benötigen wir ein paar Variablen innerhalb GitLab.
Einmal “UNITY_USERNAME” und “UNITY_PASSWORD”. Diese werden mit deinem Unity Username und Passwort belegt.
WICHTIG! Diese müssen protected sein, damit Dritte keinen Zugriff auf persönlichen Daten bekommen.
Unter dem Reiter Settings/CI_CD auf GitLab findet man die Variablen, um diese anzulegen.
Nun rerunnen wir den Job für „get-activation-file“ und der Code baut nun die Docker-Images.
Diese erzeugte nun ein Job Artifact, welches wir herunterladen und entpacken. Man besucht die offizielle Seite von https://license.unity3D.com//manual und uploadet dort die“./unity3d.alf” Datei, welche gerade entpackt wurde. Wir beantworten noch, welche Unity Edition wir nutzen und erhalten dafür unsere Lizenzdatei „Unity_v2022.x.ulf“.
Daraufhin erstellen wir eine weitere Variable auf GitLab mit dem Namen “UNITY_LICENSE” und fügen alles innerhalb der Lizenzdatei in die Variable.
Die Variablen sind wichtig, damit GitLab die Erlaubnis und Zugriffsrechte hat, um überhaupt Unity-Projekte builden zu dürfen.

3. Nun ist die Pipeline eingerichtet um Projekte bei jedem Merge zu builden, zu testen und eventuell auch zu deployen, wenn es dementsprechend eingestellt ist.

4. Da leider unsere GitLab-Runner, welche Jobs von der Pipeline nehmen und ausführen, nicht kompatibel mit
Docker+Machine als Ausführung sind, kommt es zu Fehlern beim Builden.

GUI

Für die Benutzeroberfläche (UI) unseres Spiels haben wir uns entschieden, die Logik der Buttons nicht in Unity zu implementieren, sondern direkt im Code. Das Verhalten der unterschiedlichen Buttons wird durch Methoden wie AddEventListener kontrolliert. Dies ermöglicht uns eine präzisere Steuerung und Flexibilität bei der Programmierung der Interaktionen zwischen den UI-Elementen und der Spiellogik. Indem wir die Logik der Buttons im Code verankern, können wir individuelle Funktionen und Verhaltensweisen für jeden Button definieren und anpassen.
Das Laden und Speichern des Spiels erfolgt über eine asynchrone Operation.

Networking

Wir verwenden Netcode for GameObjects Version 1.6.0.

Es agiert ein Client als Host. Jeder weitere Client verbindet sich mit dem Host.

Was ist der NetworkManager?

*Ausgegraute Attribute verwenden wir nicht.

Der NetworkManager ist das zentrale Element der Multiplayerfunktion. Er enthält alle Einstellungen die Netcode braucht, um zu funktionieren. Dort gibt es folgende Attribute:

- **LogLevel**: Setzt das network logging level
- **Player Prefab**: für jeden Client der dem Spiel joint wird eine Instanz dieses Prefabs erstellt. Das Prefab hat eine NetworkObject Component, die unter anderem festhält welchem Client diese Instanz gehört, sodass nicht jeder Client das Objekt steuern kann.
- **NetworkPrefabs**: Alle Prefabs, die während des Spiels gespawnt werden, müssen hier registriert werden. Hier sind unsere kleinen Enemies registriert, die von den großen Enemies gespawnt werden.
- **Protocol Version**: Kann verwendet werden, um Builds inkompatibel miteinander zu machen. Da wir nur ein Build haben, brauchen wir das nicht.
- **NetworkTransport**: enthält die Netzwerkeinstellungen. Wir benutzen UnityTransport.
- **TickRate:** Wie oft geupdated wird
- **Ensure Network Variable Length Safety:** (Erhöht die CPU-Verarbeitung und die Bandbreite) Wenn diese Eigenschaft aktiviert ist, verhindert Netcode for GameObjects, dass Anwendercode über die Grenzen einer NetworkVariable hinaus geschrieben wird. Ist deaktiviert.
- **Connection Approval:** Aktiviert connection approval, wenn diese Option aktiviert ist und der ConnectionApprovalCallback zugewiesen ist.

- **Client Connection Buffer Timeout:** Dieser Wert legt die Zeitspanne fest, die vergehen muss, bis ein verbindender Client den connection approval process abgeschlossen hat. Wenn die angegebene Zeit überschritten wird, wird die Verbindung des Clients getrennt. Standardwert ist 10 Sekunden. Ist nur relevant, wenn connection approvel verwendet wird.

- **Force Same Prefabs:** Wenn diese Option aktiviert ist, wird immer überprüft, ob die verbindenden Clients die gleichen registrierten Network-Prefabs haben wie der Server. Wenn diese Option nicht aktiviert ist, ignoriert Netcode for GameObjects alle Unterschiede. Ist aktiviert bei uns, damit keine Fehler beim spawnen der kleinen Enemies auftritt.

- **Recycle Network Ids:** Wenn diese Option aktiviert ist, werden zuvor zugewiesene NetworkObjectIds nach einer bestimmten Zeitspanne wiederverwendet. Bei uns deaktiviert, da wir eh nicht viele Network Objects haben.

- **Network Id Recycle Delay:** Die Zeit, die vergeht, bis eine zuvor zugewiesene, aber derzeit nicht zugewiesener identifier wieder zur Verwendung zur Verfügung steht. Nur relevant wenn Recycle Network Ids aktiviert ist.

- **Enable Scene Management:** Wenn diese Option aktiviert ist, übernimmt Netcode for GameObjects die Verwaltung der Szenen und die Synchronisierung der Clients für Sie. Bei uns aktiviert.

- **Load Scene Time Out:** Wenn "Enable Scene Management" aktiviert ist, gibt dies die Zeitspanne an, die der NetworkSceneManager wartet, während eine Szene asynchron geladen wird, bevor der NetworkSceneManager das Laden/Entladen der Szene als fehlgeschlagen/zeitlich abgelaufen betrachtet. Auf 120 Sekunden gesetzt.

Wie werden Verbindungen hergestellt?

Beim Starten eines Clients verwendet der NetworkManager die IP-Adresse und den Port, die in Ihrer Transportkomponente für die Verbindung angegeben sind. Da bei uns die Clients die Hostadresse durch das GUI eingeben, müssen wir die IP-Adresse während der Laufzeit setzen. Hierfür ist das Skript NetworkUIController zuständig. Da wir für Host und Client den gleichen Code verwenden, wird die IP-Adresse beim Start des Spiels auf die „listen address“ 0.0.0.0 gesetzt, damit der Server alle Adressen im Netzwerk abhört. Wenn dann im UI auf “Start Client” gedrückt wird, verbindet sich der Client mit der eingegebenen Adresse. Da die geladenen GameObjects später gebraucht werden, wird 2 Sekunden gewartet, bis der Code weiter ausgeführt wird.

Wie wird synchronisiert?

Es gibt 2 Autoritätsmodi bei der Synchronisation. Den Server Authoritative Mode, bei dem Änderungen des NetworkTransforms vom Server erkannt und an alle Clients geschickt werden und den Owner Authoritative Mode (dafür werden ClientNetworkTransforms

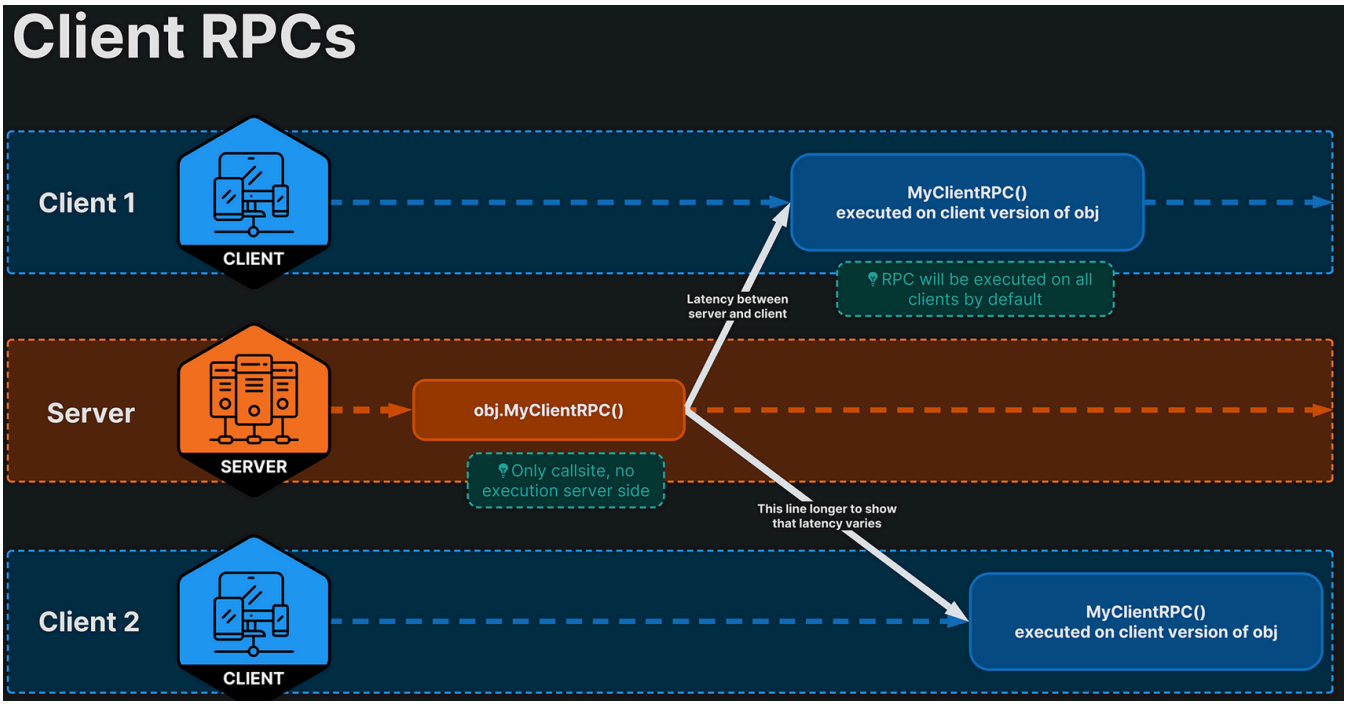
benutzt), bei dem der Besitzerclient des jeweiligen Objekts die Änderungen an den Server/Host schickt. Für die Goober(Enemies) benutzen wir das normale NetworkTransform, für den Player ClientNetworkTransform und für die Animation des Players den ClientNetworkAnimator, welcher die Animationen synchronisiert. Das heißt Player sind Owner Authoritative und Enemies Server Authoritative.

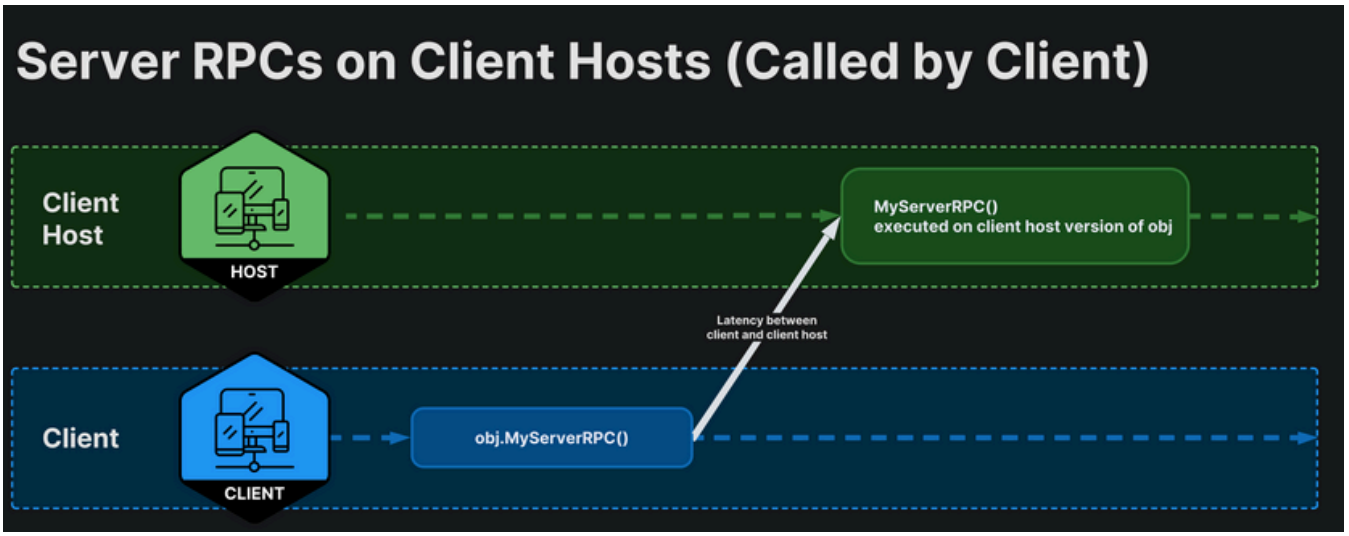
Dadurch, dass Enemies Owner Authoritative sind, können Clients ihre Werte nicht verändern. Das würde heißen, Clients können keine Enemies töten. Um das Problem zu umgehen, verwenden wir Remote Procedure Calls, kurz: **RPCs**. RPCs können auf NetworkObjects aufgerufen werden. Der RPC wird dann in die lokale Warteschlange gestellt und dann am Ende des Frames an den Server gesendet. Nach dem Empfang des Server-RPCs wird dieser auf der Instanz des NetworkObjects des Servers ausgeführt. Dies verwenden wir, um den verursachten Schaden an den Server zu schicken, der dann dem Enemy Leben abzieht. Die gleiche Logik verwenden wir beim endgültigen Töten der Enemies und beim Verleihen von Fähigkeiten nach dem Töten von Bossen, da beide Spieler die Fähigkeiten gegeben werden müssen, obwohl ja nur einer den letzten Hit ausführt.

Timing Diagramme zur Visualisierung:

Bei jeder neuen Verbindung führt Netcode for GameObjects (Netcode) einen Handshake durch, zusätzlich zu den Handshakes, die der Transport durchführt. Dadurch wird sichergestellt, dass die Netzwerkkonfiguration auf dem Client mit der Netzwerkkonfiguration des Servers übereinstimmt.

Da wir kein NetworkManager.ConnectionApprovalCallback benutzen, verwendet Netcode basic authentication für den Benutzer, die automatisch den default **Player Prefab** autorisiert und zuweist.





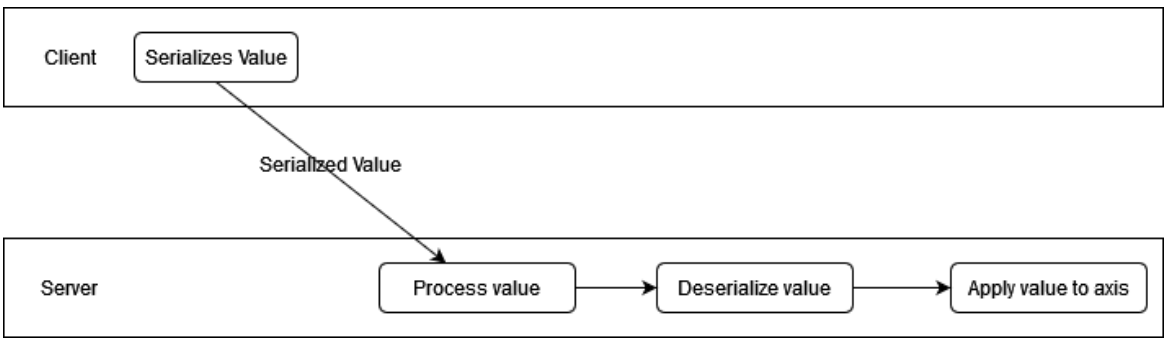
Transport

Als Transport layer benutzen wir das Unity Transport Package, welches das default package ist.

Da der Kommunikationsfluss nicht überall gleich ist, sind hier einige Beispiele:

Andere Beispiele zum Kommunikationsfluss:

- Laden neuer Szenen: https://docs-multiplayer.unity3d.com/img//sequence_diagrams/SceneManagement/SwitchingToNewScene_Light.png?text=LightMode
- ClientNetworkTransform (Position des eigenen Spielers synchronisieren)
    - Passiert immer wenn sich der Wert ändert, bzw nach dem eingestellten threshold
    - Beim NetworkTransform sind Client und Server vertauscht
- Der Aufbau der Connection erfolgt per TCP handshake



Quellen:

- https://docs-multiplayer.unity3d.com/netcode/1.6.0/about/
- https://docs.unity3d.com/Packages/com.unity.transport@2.2/manual/

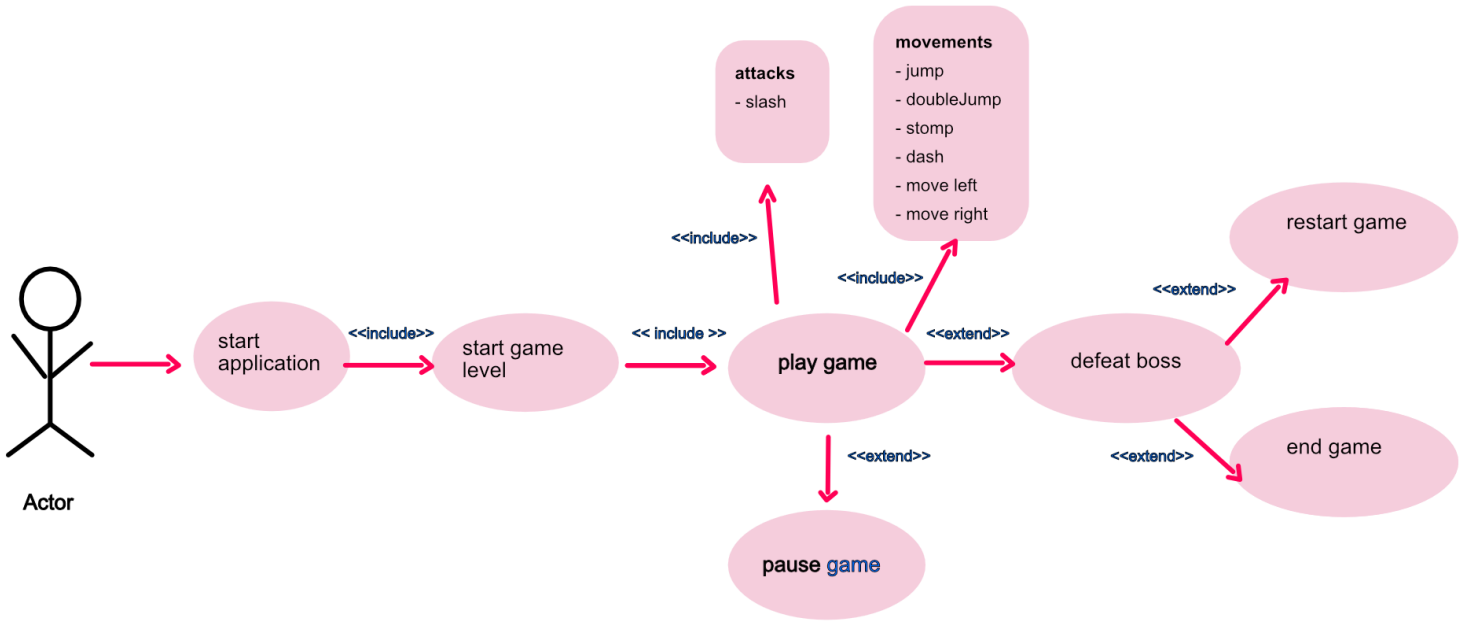
Persistenz

SaveSystem iteriert über alle Skripte drüber die das SaveandLoad Interface implementiert haben. Danach werden die Daten über einen Stream in eine Liste im JSON-Format gespeichert. Deserialization funktioniert genau rückwärts. Es werden Dinge wie Score im SaveSystem verarbeitet und gespeichert.

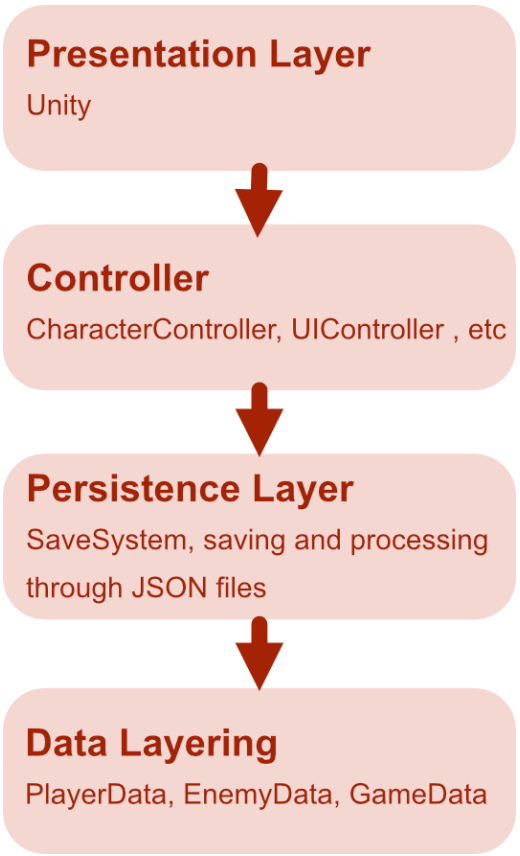
Abschlusspräsentation/Endgabe Reflexion

Name	**Was habe ich gelernt**	**Wo habe ich noch Lücken**	**Bei diesen Themen Bin ich über meinen Schatten gesprungen**
Lela Grösser	Netcode, im Team arbeiten, Was Pipelines machen	Bei Unity Software Architektur/Game Architektur	Teamarbeit
Julia Breuer	Testing mit Unity Framework, GUI Logik (Panels)	Mit einer Game Engine arbeiten	Merge Konflikten lösen, Git über Konsole bedienen
Erik Lindinger	Save System, Pipelines erstellen	Clean Code, Testing, Saubere commits, Aufgabenverteilung	CI/CD, Merge Konflikte in scenes und prefabs lösen
Jonathan Kaufmann	Mit dem Editor von Unity zu arbeiten	Bei der Dokumentation	Git über Konsole
Hana Le	Mit Unity zu arbeiten, GUI Logik, C#	Mit einer Game Engine arbeiten, mit Git zu arbeiten besonders Merge conflicts und noch Unity	Git über Konsole bedienen und Teamarbeit

Use Case Diagramm



Architekturdiagramm



UML

Siehe Repo